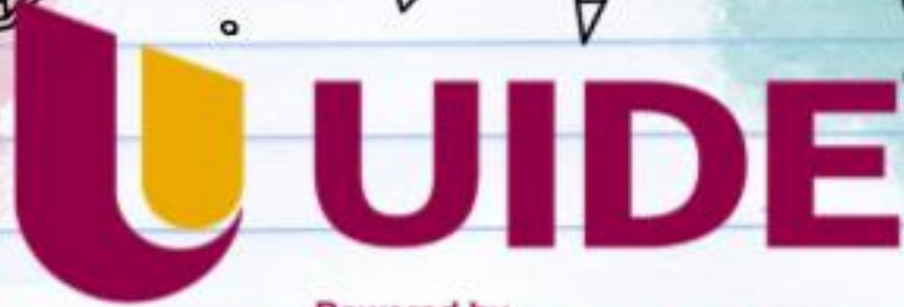




Powered by
Arizona State University®

LOGICA DE PROGRAMACION

Desarrollo de Software: Generador Seguro
de Contraseñas

Hirvin Luis Bravo Garcia

Contenido

1. Introducción.....	3
PASO 1.	3
1.1 Diversos especialistas en ciberseguridad recomiendan:	3
2. Análisis del Problema	4
3. Posición Personal.....	4
4. Objetivo del Software	4
5. Requerimientos Funcionales	4
5.1 El sistema deberá permitir:.....	4
6. Configuración del Entorno de Desarrollo	4
7. Conclusiones del Análisis.....	5
8. DESARROLLO DEL PROGRAMA	5
8.1 Descripción General	5
10. Funcionamiento.....	6
11. Reflexión sobre el Desarrollo	6
11.1 Pensamiento Innovador	6
12. Vinculación y Síntesis	6
13. Conclusiones Generales	6
14. Anexos	7

Desarrollo de Software:

Generador Seguro de Contraseñas

1. Introducción

La seguridad informática se ha convertido en una necesidad fundamental dentro de la sociedad digital actual. La mayoría de los servicios utilizados diariamente, como redes sociales, plataformas educativas, aplicaciones bancarias y sistemas empresariales, requieren mecanismos de autenticación para proteger la información de los usuarios.

Uno de los problemas más frecuentes es la utilización de contraseñas débiles o fáciles de adivinar, lo que incrementa significativamente el riesgo de accesos no autorizados y ataques informáticos.

Por esta razón se propone el desarrollo de un Generador Seguro de Contraseñas, capaz de crear claves robustas mediante la combinación aleatoria de letras, números y caracteres especiales.

PASO 1.

INICIO DEL DESARROLLO DE SOFTWARE Y CONFIGURACIÓN DEL ENTORNO

Investigación

Para el desarrollo de esta solución se realizó una investigación sobre las mejores prácticas utilizadas en seguridad informática para la generación de contraseñas.

1.1 Diversos especialistas en ciberseguridad recomiendan:

- Utilizar contraseñas de al menos 12 caracteres.
- Combinar letras mayúsculas y minúsculas.
- Incorporar números.
- Incorporar símbolos especiales.
- Evitar información personal fácilmente identificable.

De acuerdo con organismos especializados en seguridad informática, las contraseñas complejas reducen significativamente la probabilidad de ataques por fuerza bruta y diccionario.

Sin embargo, algunos expertos argumentan que la complejidad excesiva puede dificultar que los usuarios recuerden sus credenciales. Por ello, el diseño de esta aplicación busca encontrar un equilibrio entre seguridad y facilidad de uso.

2. Análisis del Problema

Actualmente muchos usuarios crean contraseñas utilizando:

- Fechas de nacimiento.
- Nombres propios.
- Números consecutivos.
- Palabras comunes.

Estas prácticas generan vulnerabilidades que pueden ser explotadas por atacantes.

El problema principal consiste en proporcionar una herramienta sencilla que genere automáticamente contraseñas seguras sin depender de la creatividad o experiencia técnica del usuario.

3. Posición Personal

Considero que la mejor solución es automatizar el proceso de creación de contraseñas mediante un software especializado.

La automatización elimina errores humanos y garantiza que las claves generadas cumplan con estándares mínimos de seguridad.

Reconozco que ninguna contraseña es completamente invulnerable; sin embargo, una contraseña generada aleatoriamente ofrece un nivel de protección significativamente superior a una creada manualmente por el usuario promedio.

4. Objetivo del Software

Desarrollar una aplicación capaz de generar contraseñas seguras y aleatorias que permitan fortalecer la seguridad de los usuarios en diferentes plataformas digitales.

5. Requerimientos Funcionales

5.1 El sistema deberá permitir:

1. Seleccionar la longitud de la contraseña.
2. Incluir letras mayúsculas.
3. Incluir letras minúsculas.
4. Incluir números.
5. Incluir caracteres especiales.
6. Generar una contraseña aleatoria.
7. Mostrar el resultado al usuario.

6. Configuración del Entorno de Desarrollo

Para el desarrollo del proyecto se utilizarán las siguientes herramientas:

- Lenguaje de programación: Python
- Editor de código: Visual Studio Code
- Sistema operativo: Windows
- Consola de ejecución integrada de Python

Estas herramientas fueron seleccionadas por su facilidad de uso y amplia adopción en la industria del software.

7. Conclusiones del Análisis

Después de analizar diferentes alternativas, se concluye que un generador de contraseñas es una solución práctica y eficiente para mejorar la seguridad digital.

La investigación realizada demuestra que la automatización del proceso de generación reduce considerablemente el riesgo de utilizar contraseñas débiles.

Asimismo, la implementación en Python facilita el desarrollo y mantenimiento del software gracias a su sintaxis sencilla y amplia disponibilidad de bibliotecas.

PASO 2.

8. DESARROLLO DEL PROGRAMA

8.1 Descripción General

El programa solicita al usuario la longitud deseada para la contraseña y posteriormente genera una combinación aleatoria de caracteres.

La contraseña resultante puede contener:

- Letras mayúsculas.
- Letras minúsculas.
- Números.
- Símbolos especiales.

9. Código Fuente

```
python

import random

import string

longitud = int(input("Ingrese la longitud de la contraseña: "))

caracteres = string.ascii_letters + string.digits + string.punctuation

contrasena = ""

for i in range(longitud):

    contrasena += random.choice(caracteres)

print("Contraseña generada:")

print(contrasena)
```

10. Funcionamiento

1. El usuario ejecuta el programa.
2. Introduce la longitud deseada.
3. El sistema construye una lista de caracteres válidos.
4. Se seleccionan caracteres aleatorios.
5. Se genera la contraseña.
6. El resultado se muestra en pantalla.

11. Reflexión sobre el Desarrollo

Durante el desarrollo fue necesario analizar diferentes formas de generar cadenas aleatorias.

Inicialmente se consideró utilizar únicamente letras y números; sin embargo, se decidió incorporar símbolos especiales debido a que incrementan significativamente la seguridad de la contraseña.

También se evaluó la posibilidad de utilizar una interfaz gráfica, pero se optó por una aplicación de consola para simplificar el desarrollo y enfocarse en la lógica del programa.

Estas decisiones permitieron crear una solución funcional, eficiente y fácil de comprender.

11.1 Pensamiento Innovador

La aplicación puede evolucionar en futuras versiones incorporando:

- Medidor de fortaleza de contraseña.
- Historial de contraseñas generadas.
- Exportación a archivos.
- Interfaz gráfica amigable.
- Integración con gestores de contraseñas.

Estas mejoras ampliarían la utilidad del software y proporcionarían una mejor experiencia al usuario.

12. Vinculación y Síntesis

El proyecto integra conocimientos de:

- Lógica de programación.
- Seguridad informática.
- Generación aleatoria de datos.
- Diseño de software.
- Resolución de problemas.

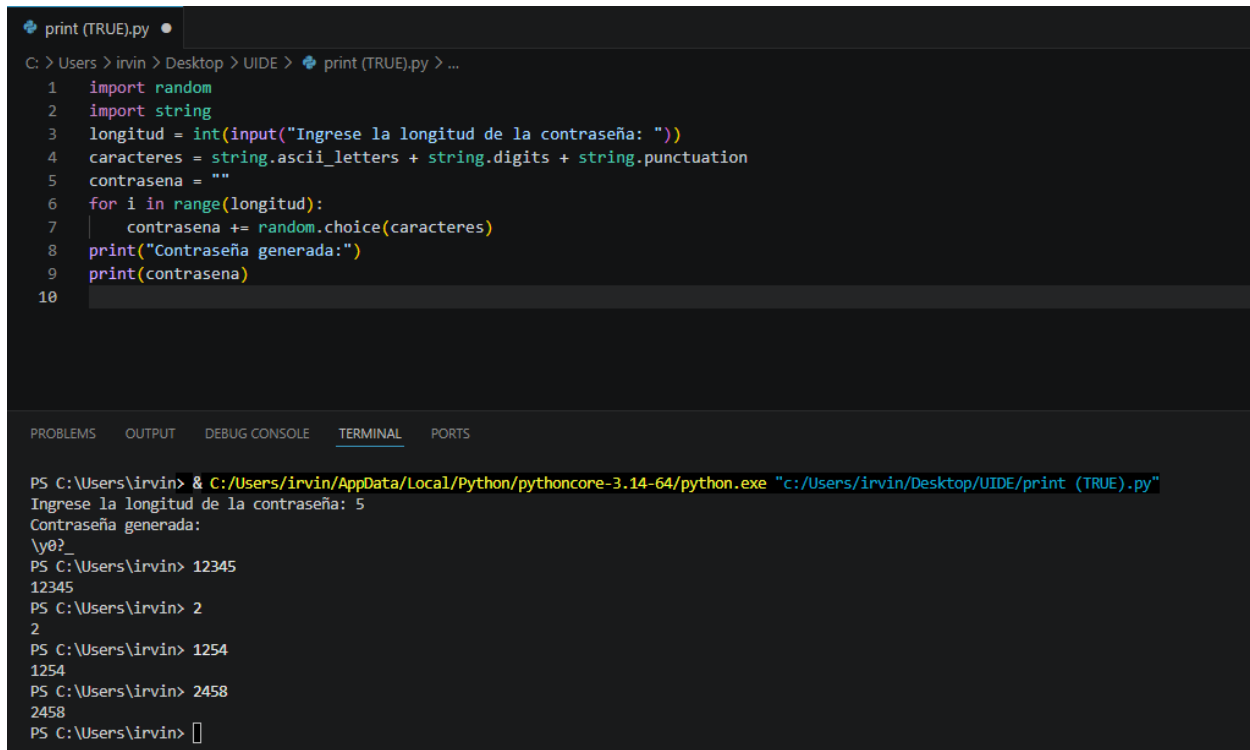
La combinación de estos elementos permitió transformar una necesidad cotidiana en una solución tecnológica funcional.

13. Conclusiones Generales

1. La programación permite desarrollar soluciones prácticas para problemas reales.
2. La generación automática de contraseñas mejora la seguridad de los usuarios frente a amenazas informáticas.

3. Python es una herramienta adecuada para el desarrollo de aplicaciones debido a su simplicidad y potencia.
4. El análisis previo del problema facilita la construcción de software más eficiente.
5. La integración de conceptos de programación y seguridad informática genera soluciones innovadoras y aplicables en distintos contextos.
6. El proyecto demuestra la importancia de la planificación y el diseño antes de iniciar la implementación del código.

14. Anexos



The image shows a code editor with a Python script named `print (TRUE).py`. The script generates a random password of a specified length. Below the code editor, the terminal window shows the execution of the script, including the prompt for password length and the resulting password.

```
print (TRUE).py
C: > Users > irvin > Desktop > UIDE > print (TRUE).py > ...
1  import random
2  import string
3  longitud = int(input("Ingrese la longitud de la contraseña: "))
4  caracteres = string.ascii_letters + string.digits + string.punctuation
5  contrasena = ""
6  for i in range(longitud):
7      contrasena += random.choice(caracteres)
8  print("Contraseña generada:")
9  print(contrasena)
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\irvin> & C:/Users/irvin/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/irvin/Desktop/UIDE/print (TRUE).py"
Ingrese la longitud de la contraseña: 5
Contraseña generada:
\y0?_
PS C:\Users\irvin> 12345
12345
PS C:\Users\irvin> 2
2
PS C:\Users\irvin> 1254
1254
PS C:\Users\irvin> 2458
2458
PS C:\Users\irvin>
```